
Scientific Tacit Knowledge Capture with Vision Models

Joe Tey
Stanford University
joetey@stanford.edu

Abstract

We build a system that predicts which well(s) of a 96-well plate a pipette dispenses into, from a paired first-person (FPV) and top-down video clip. With only ~ 100 labeled clips across 7 plates, treating this as 96-way classification is infeasible. We instead reframe the task as *geometry*: a three-stage pipeline detects the dispensing frame, detects the plate’s four corners to recover a homography into a canonical grid, and localizes the pipette tip in that canonical frame—quantizing to a well only at the very end. Under leave-one-plate-out (LOPO) evaluation the pipeline reaches **27% exact-match (32% on single-channel clips)** at **\$0** marginal cost and **3.5 s/clip** on CPU, versus **2–4%** for a zero-shot frontier multimodal large language model (MLLM; Gemini 3.1 Pro) at \$0.03–0.04/clip and 21–26 s/clip. A per-stage error budget attributes the system’s ceiling to plate-corner detection under occlusion (the multi-channel pipette head hides the plate), and a latency breakdown shows the runtime is video-decode-bound, not model-bound.

1 Background and Setup

Motivation. Most laboratory work is unobserved: a protocol records what was *intended*, but the actual execution—which wells were touched, in what order—survives only as tacit know-how in a researcher’s hands, a root cause of the reproducibility crisis. Software made opaque execution legible through *observability* (logs and traces); the wet lab has no equivalent, seeing inputs and outputs but never the trace in between. We treat “which well did the pipette dispense into” as the atomic event of that missing trace—a semantic, machine-readable record recovered passively from ordinary video—so that stacking these events yields an execution log of a physical experiment. This paper delivers that primitive.

Task, inputs, and outputs. The input is a pair of time-synchronized clips of one pipetting action: an FPV camera and a top-down camera, each 1920×1080 at 30 fps, ~ 71 frames (~ 2.4 s). The output is the dispensed well(s) as a list of {well_row, well_column} pairs. A single-channel pipette dispenses into one well; an 8-channel pipette into a full column (8 wells); a 12-channel pipette into a full row (12 wells).

Goals. (G1) **Exact-match** the labeled well—both row and column. (G2) **Generalize to unseen plates**: the eventual holdout is new clips, so the metric that matters is leave-one-plate-out, not on-data accuracy. (G3) **Handle all pipette modes** (single, 8-channel, 12-channel).

Constraints. These drive every design decision; the first is fundamental. (C1) **Data scarcity**: only 100 clips over 7 plates—roughly one example per well. (C2) **The output is a metric 8×12 grid, not a flat label set**: legal answers are grid cells or head-aligned full rows/columns, and the natural error metric is grid distance, not 0/1. (C3) **Minimize human labeling**: a budget of under one hour of annotation. (C4) **Near-zero marginal inference cost**. (C5) **Runtime**: the full 100-clip evaluation must finish well under 20 minutes on commodity hardware.

Hypothesis (the falsifiable question). *In a data-scarce regime, exploiting the geometric inductive bias of the rigid 96-well plate generalizes to unseen plates better, per unit of labeling effort and inference cost, than approaches that predict the well directly in label space—a frontier multimodal*

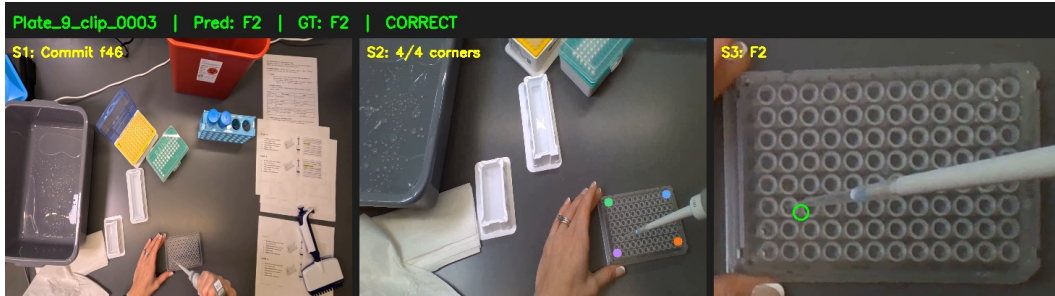


Figure 1: The three-stage pipeline on one clip (Plate_9_clip_0003, prediction F2 = ground truth). **Stage 1:** a Random Forest selects the *commit frame* (dispensing moment, frame 46) from the top-down view. **Stage 2:** a YOLOv11n-pose model detects the four plate corners in the FPV commit frame (colored dots), defining a homography. **Stage 3:** the FPV frame is warped to a canonical plate view and a CNN localizes the tip (green circle), which quantizes to F2.

large language model (MLLM) or a simple classifier. The intuition: with ~ 1 example per well you cannot learn a 96-way output space, but you *can* learn the transferable sub-skills “find the four corners” and “find the pipette tip,” whose difficulty is independent of *which* well is involved. The hypothesis is falsifiable: if a zero-shot frontier MLLM matches the geometric pipeline at comparable cost, it is rejected.

Crux of the technical problem. The hard part is the collision of C1 and C2: a structured 96-class output learned from ~ 100 examples. The crux is realizing that the structure that makes the problem hard (the rigid grid) is also what makes it solvable—if we stop predicting the *label* and instead predict a *continuous geometric quantity* (the tip’s position in a plate-induced coordinate frame) from which the label is computed. Every clip then feeds one shared geometric estimator instead of being spread thin across 96 classes.

2 Approach (Method)

Organizing principle. We never predict the well directly. We recover the plate’s pose from its four corners, use it to rectify the scene into a *canonical grid coordinate frame* in which every well sits at a fixed, known location, localize the pipette tip in that frame, and quantize to a well only at the final step. Concretely this is a three-stage pipeline (Figure 1); we summarize each stage here and defer hyperparameters and architecture to Appendix A.

Stage 1 — Commit-frame detection (when). A Random Forest classifies each top-down frame as “dispensing” or not, from per-frame motion features (how still is the frame; how much motion occurred in the surrounding ± 5 –30 frames) and MediaPipe hand-skeleton features. We use the top-down view here because it cleanly captures the hand pausing to dispense. *Design decision:* a cheap Random Forest on hand-engineered features rather than a learned temporal network, because C3 affords only a start/end window per clip and a heavy temporal model would overfit ~ 100 windows. The most informative features are the *backward* motion windows (37% of importance; Appendix A)—the dispensing pause is best identified by the motion that follows it (the withdrawal).

Stage 2 — Plate-corner detection (where is the grid). A YOLOv11n-pose model (2.66M parameters) detects the four corners (A1, A12, H12, H1) in the FPV commit frame; these define the homography into the canonical frame. *Design decisions:* (i) we use the FPV view, not top-down, because the top-down camera is too low-resolution to resolve individual wells; (ii) corners are the one quantity that genuinely *must* be labeled—they cannot be derived—but they *amortize* (the same rigid object in every clip of a plate) and transfer across plates; (iii) we use the *nano* model, since with ~ 100 images larger variants overfit.

Stage 3 — Tip localization + pipette type (which well). On the rectified image a two-headed CNN predicts (a) a heatmap whose peak is the pipette tip, at a spatial resolution matched to the well grid, and (b) a pipette-type label (single / 8-channel / 12-channel). The heatmap peak maps to a canonical coordinate and quantizes to the nearest well; the type head expands a single anchor well to the full row or column. *What did not work:* we first tried a YOLO keypoint model for the tip, but the

Table 1: Dataset. Crucially, each plate contains exactly one pipette type—type is *perfectly confounded with plate*, which matters for the multi-channel analysis.

Plate	Type	Clips
Plate 1	multi-col (full row, 12-ch)	6
Plate 2	multi-row (full col, 8-ch)	12
Plate 3	single pipette	11
Plate 4	single pipette	20
Plate 5	multi-col (full row, 12-ch)	7
Plate 9	single pipette	23
Plate 10	single pipette	21
Total (75 single, 13 multi-col, 12 multi-row)		100

transparent tip is barely visible against the plate and YOLO failed; a heatmap CNN, which predicts a soft spatial distribution rather than a hard keypoint, was far more robust. *Key design decision:* the heatmap is supervised with the *manually-labeled tip*, not the homography-derived well center—the latter is a fixed pixel determined only by the label, so training on it would silently collapse Stage 3 into a 96-way classifier.

What we started with. Off-the-shelf YOLOv11n-pose, scikit-learn RandomForest, MediaPipe, and Google’s Gemini 3.1 Pro API (for baselines). We built: a browser-based keypoint labeler, all three stage models and their training, the homography/quantization logic, and a unified LOPO evaluation harness that scores every method through one scorer while logging accuracy, per-stage latency, and token cost.

3 Evaluation and Results

Definition of success. Success is supporting the hypothesis (§1): the geometric pipeline should beat a strong non-geometric approach on LOPO exact-match (G1, G2) *at lower labeling and inference cost* (C3, C4), while handling all pipette modes (G3). We therefore measure, under one matched protocol: exact-match and tolerance accuracy, per-stage accuracy (to attribute the result), \$/clip, and latency.

Experimental setup. *Dataset:* 100 clips, 7 plates (Table 1). *Protocol:* leave-one-plate-out for our pipeline (each clip predicted by models that never saw its plate); the MLLM baselines are zero-shot, so LOPO is automatic. Plates differ markedly in lighting and background clutter, so LOPO is a genuine test of appearance generalization, not just held-out clips of a familiar scene. *Machine:* a single CPU core (CUDA disabled), since the pipeline is not GPU-bound (§3.4). *Baselines.* Both are **zero-shot Gemini 3.1 Pro** (the current vision leader), FPV-only, given *raw* frames—no warped images and no oracle corners, so the model must recover the geometry itself. A shared system prompt supplies the domain facts (8 rows A–H, 12 columns 1–12, the A1 corner/bevel convention, and the three pipette types), and a forced JSON schema returns (pipette_type, well_row, well_column) which we expand to the full row/column with the *same* logic as our pipeline, so scoring is identical. Both use an explicit 5-step chain-of-thought (orientation → commit moment → tip location by counting columns/rows from a reference edge → pipette type → answer). They differ in structure: **Baseline A (monolithic)** makes *one* call with 16 uniformly sampled frames, asking it to find the dispensing frame and read the well together; **Baseline B (decomposed)** makes *two* calls—Call 1 picks the dispensing frame index from the 16 frames, Call 2 reads the well from that single *full-resolution* frame, mirroring our pipeline’s stage decomposition and spending the resolution budget where it matters. Full prompts are in Appendix B.

3.1 Main result: geometry vs. a frontier MLLM (G1, G2)

Table 2 and Figure 2 are the central result, and they **support the hypothesis**. The geometric pipeline reaches **27%** exact-match; the strongest zero-shot frontier model we could fairly build reaches **2–4%**—a 7–13 $\times$ gap—while costing money per clip and running 6–8 $\times$ slower.

Interpreting the gap: it is precise column counting. The aggregate number hides *where* the MLLM fails. Decomposing by axis, the MLLM gets rows roughly right (33–34%) but collapses on columns (9–14%), whereas our pipeline reaches 36% on columns. We speculate this row/column

Table 2: End-to-end comparison (100 clips, LOPO / zero-shot, FPV-only). Exact / Within-1 / Within-2 are cell accuracy at Chebyshev tolerance 0/1/2.

Method	Exact	Within-1	Within-2	\$/clip	Latency/clip
My Approach (3-stage)	27/100	50/100	58/100	\$0	3.5 s
Baseline A (monolithic)	2/100	15/100	30/100	\$0.034	20.9 s
Baseline B (decomposed)	4/100	23/100	39/100	\$0.041	25.7 s

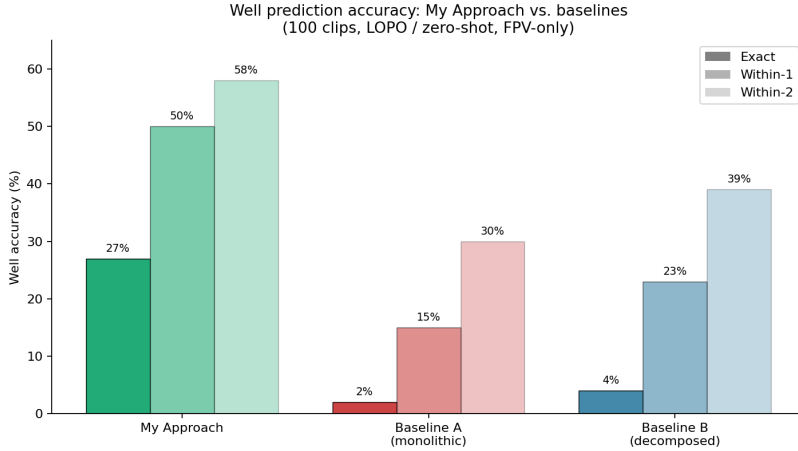


Figure 2: Accuracy at increasing tolerance. The gap is largest at Exact and narrows but never closes at looser tolerance—the MLLM is “in the neighborhood” but cannot count cells precisely.

asymmetry is itself geometric: there are only 8 rows but 12 finer columns, and the oblique FPV viewpoint foreshortens the long (column) axis of the plate far more than the short (row) axis—so counting columns demands undoing exactly the perspective distortion that a homography corrects for free and a MLLM must eyeball. The MLLM clearly *locates* the plate and the rough region (Within-2 is 30–39%, so it is “in the neighborhood” a third of the time) but cannot *count cells precisely*—exactly the gap a homography fills, by turning “count seven columns under perspective distortion” into an exact coordinate lookup. The narrowing-but-never-closing tolerance curve in Figure 2 is the visual signature of this. Decomposing the MLLM (B > A: 4 vs. 2 exact) helps—resolution and focus matter—but does not save it: the limiting factor is *perceptual precision*, not task structure. So the obvious simpler approach, “hand the whole clip to a frontier model,” fails precisely on the sub-cell geometry the pipeline was built for.

3.2 Per-stage results and the error budget (attribution)

We isolate each stage to attribute the end-to-end number. Stage 1 measures whether the predicted frame falls inside the labeled dispensing window. Stage 2 fixes an oracle tip and measures well accuracy, isolating corner/homography quality. Stage 3 fixes oracle corners and measures well accuracy, isolating tip detection. All are LOPO. Tables 3–5 give per-plate detail; Table 6 composes them.

Two readings matter. **First**, the naive product ($0.80 \times 0.696 \times 0.733 \approx 41\%$) *overestimates* the observed 27%. We interpret this 14-point gap as evidence that the stages are not independent: each isolated accuracy is measured on the (easier) subset where the other stages succeed or are oracle-supplied, and genuinely hard clips—occluded, oblique—tend to fail multiple stages at once, so errors correlate rather than compound cleanly. We state this as interpretation, not a proven decomposition. **Second, the bottleneck is Stage 2 (corners) at 69.6%**: Stage 1 is reliable (80%) and Stage 3 given good corners is comparable (73.3%). The actionable conclusion is that the next labeling and modeling effort should target *corner detection*, not the tip stage.

Table 3: Stage 1: % predicted frame in-window.

Plate	Type	In-window
Plate 1	multi-col	6/6 (100%)
Plate 2	multi-row	11/12 (92%)
Plate 3	single	8/11 (73%)
Plate 4	single	18/20 (90%)
Plate 5	multi-col	7/7 (100%)
Plate 9	single	14/23 (61%)
Plate 10	single	16/21 (76%)
All		80/100 (80%)

Table 4: Stage 2: well acc. | oracle tip (isolates corners).

Plate	Type	Well acc.
Plate 1	multi-col	1/6 (16.7%)
Plate 2	multi-row	3/11 (27.3%)
Plate 3	single	6/11 (54.5%)
Plate 4	single	12/15 (80.0%)
Plate 5	multi-col	1/5 (20.0%)
Plate 9	single	23/23 (100%)
Plate 10	single	18/21 (85.7%)
All		64/92 (69.6%)

Table 5: Stage 3: well acc. | oracle corners (isolates tip).

Plate	Type	Well acc.
Plate 1	multi-col	4/6 (66.7%)
Plate 2	multi-row	6/11 (54.5%)
Plate 3	single	8/11 (72.7%)
Plate 4	single	5/15 (33.3%)
Plate 5	multi-col	6/7 (85.7%)
Plate 9	single	22/23 (95.7%)
Plate 10	single	12/13 (92.3%)
All		63/86 (73.3%)

Table 6: Error budget. The product of isolated-stage accuracies overestimates the observed end-to-end number.

Quantity	Accuracy	Stages are not independent
Stage 1 (commit frame)	80.0%	
Stage 2 (corners tip)	69.6%	
Stage 3 (tip corners)	73.3%	
Naive product	$\approx 41\%$	
Observed end-to-end	27%	

independent: each isolated accuracy is measured where the other stages are oracle/easy, and hard clips fail several stages together, so errors correlate rather than compound cleanly.

3.3 Why multi-channel fails: occlusion (G3)

Table 7 shows the system does poorly on multi-channel, and the per-stage tables tell us *why*. In Stage 2 (Table 4) the multi-channel plates (1, 2, 5) score 17–27% while single-channel plates score 55–100%—corner detection is where multi-channel collapses. Crucially, when corners are oracle-supplied (Table 5) multi-channel recovers to 55–86%, so the tip stage is *not* the problem. The cause is **occlusion**: a multi-channel pipette has a wide multi-tip head that, together with the operator’s hand, hides part of the plate. As Figure 3 shows, Stage 2 still reports “4/4 corners detected,” but the detector places the occluded corners on the pipette body or collapses the quad onto the visible half—so the homography maps the wrong region and every downstream well is wrong. A secondary, honest caveat is the *plate-type confound* (Table 1): multi-row appears on only one plate, so LOPO removes all multi-row training when that plate is held out, which inflates its difficulty. The clean, non-confounded headline is therefore **single-pipette accuracy, 24/75 = 32%**.

3.4 Cost, latency, and labeling (C3, C4, C5)

Cost (C4). The pipeline is \$0 marginal after a one-time training cost of a few minutes; the MLLMs cost \$0.034–0.041/clip *forever* (Figure 4b). At a realistic lab scale of 1,000 dispenses/day that is \$0 vs. $\sim$ \$12.5k–14.8k/year, and the pipeline runs offline and deterministically with no network dependency. **Runtime (C5).** 100 clips finish in $\sim$ 5 minutes on a single CPU core (3.5 s/clip), well under the 20-minute budget. **Why is the latency what it is? Not the models.** Figure 4a attributes the 3.45 s/clip: **Stage 1 = 3.01 s (87%)**, Stage 2 = 0.16 s, Stage 3 = 0.05 s, plus $\sim$ 0.23 s overhead. The networks are nearly free; the cost is *decoding the entire top-down video* to compute Stage-1 motion features. We confirmed a CPU-only run is no slower than a GPU run—the Random Forest cannot use the GPU and the nano networks finish in milliseconds—so *the pipeline is I/O-bound (video decode), not compute-bound*. The optimization lever is to sample frames in Stage 1, not to buy faster hardware: a useful negative result, since the reflexive “put it on the GPU” would have bought nothing. **Labeling (C3).** The final pipeline is trained from *one annotated frame per clip*: a dispensing start/end window, 4 corners, and 1 tip, across 100 clips—comfortably under one hour, and roughly an order of magnitude less than the dense per-frame keyframe datasets we abandoned.

Table 7: End-to-end accuracy by pipette type (My Approach, LOPO).

Type	Clips	Exact	Within-1	Within-2
Single pipette	75	24/75 (32.0%)	44/75 (58.7%)	48/75
Multi-row	12	0/12 (0.0%)	2/12 (16.7%)	6/12
Multi-col	13	3/13 (23.1%)	4/13 (30.8%)	4/13



(a) Multi-col (12-channel). The blue pipette head occludes the right of the plate, so the “A12/H12” corners are placed *on the pipette*, not the plate.

(b) Multi-row (8-channel). The head and hand hide the right columns, so the corner quad covers only the visible left portion of the plate.

Figure 3: Occlusion is the dominant multi-channel failure. In both cases Stage 2 reports “4/4 corners detected,” but the wide multi-tip head occludes part of the plate, so the corners—and hence the homography—are wrong.

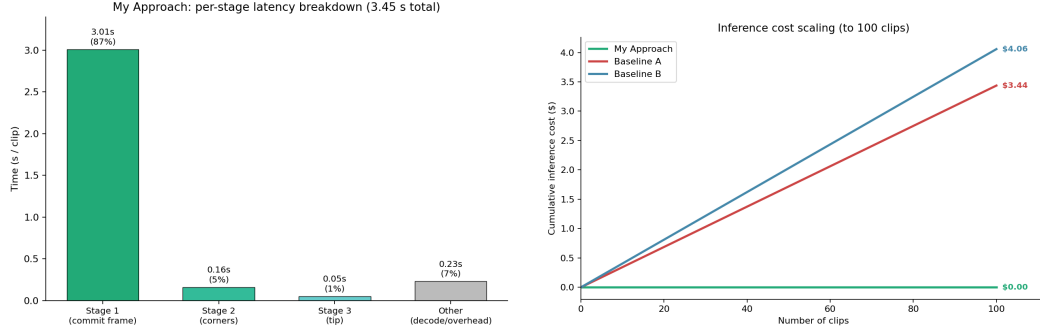
3.5 Discussion

The hypothesis is supported: under data scarcity, an explicit geometric decomposition beats a frontier MLLM by 7–13 $\times$ on exact-match while costing nothing per clip and running 6–8 $\times$ faster, with the advantage concentrated in the precise column-counting that the metric prior provides. *What worked* was the central reframing (classification $\rightarrow$ calibrated coordinate regression) and giving each stage a transferable sub-skill. *What did not work*: a YOLO keypoint model for the near-invisible tip (replaced by a heatmap CNN), and—per the error budget—corner detection under occlusion, which is the system’s true ceiling. *Could a simpler approach have done better?* On this evidence, no: the simplest strong alternative (a frontier MLLM on raw clips) fails on exactly the geometry the pipeline handles. The honest caveats are that the 27% headline is confounded by single-channel-only plates for the multi-channel types (we recommend the 32% single-pipette figure), that the per-stage budget is a sanity estimate rather than a clean decomposition, and that our MLLM baselines are deliberately *fair* (zero-shot, no warped input) rather than maximal. The clearest path forward is occlusion-robust corner detection with more corner supervision, plus a data-collection fix—mixing pipette types within plates—to de-confound the multi-channel evaluation.

A Training details

Stage 1 (Random Forest). 500 trees, max depth 20, min samples per leaf 5; input is a single frame’s feature vector, output is a binary “valid dispensing frame” label. Features (with category importances): backward motion windows (20 features, 37.2% importance), hand skeleton (61, 20.8%), other motion/position (21, 16.2%), forward motion windows (24, 13.5%), and pre/post contrast (20, 12.3%). Motion features capture stillness and surrounding motion over 5–30 frame windows; hand features come from MediaPipe (wrist and centroid position, finger spread/curl, palm width, grip geometry, and hand displacement over up to 130 frames).

Stage 2 (YOLOv11n-pose). 2.66M parameters, 6.7 GFLOPs. Trained 200 epochs (early stop patience 20), image size 1280, batch 8, AdamW at lr 0.002. Keypoints: 4 corners (A1, A12, H12, H1) with $\sigma = 0.05$ and pose loss weight 12.0; horizontal-flip keypoint mapping [1, 0, 3, 2] (A1 $\leftrightarrow$ A12, H1 $\leftrightarrow$ H12). Augmentation: flip 0.5, rotate $\pm 5^\circ$, translate 0.1, scale 0.3, no mosaic. Each manual corner label is propagated to ± 2 neighboring frames within the dispensing window (one click $\rightarrow$ five training images, as free augmentation).



(a) Per-stage latency. The total is dominated by Stage 1 (video decoding), not the neural networks. (b) Cumulative inference cost. Our pipeline is \$0 marginal; the MLLMs pay per call, forever.

Figure 4: Latency attribution (left) and cost scaling (right).

Stage 3 (two-headed CNN). Input $3 \times 240 \times 320$, ImageNet-normalized. A shared encoder (Conv $3 \rightarrow 32 \rightarrow 32$, pool; $32 \rightarrow 64 \rightarrow 64$, pool; $64 \rightarrow 128 \rightarrow 128 \rightarrow 128$) feeds (a) a heatmap head (Conv $128 \rightarrow 64 \rightarrow 32 \rightarrow 1$) producing a $1 \times 60 \times 80$ map whose patch size matches the well grid, and (b) a classification head (AdaptiveAvgPool $\rightarrow$ FC $128 \rightarrow 3$) producing single / multi-col / multi-row logits. Augmentation: horizontal flip, brightness shift (± 20), contrast scale ($0.8\text{--}1.2\times$), Gaussian noise ($\sigma = 8$), spatial translation (± 10 px), Gaussian blur (kernel $3/5/7$), and channel shuffle.

Baselines (Gemini 3.1 Pro). Cost is computed from logged input, output, and thinking tokens at published per-token prices. The exact prompts are below.

B Baseline prompts

Shared system prompt (all calls).

You are an expert at reading 96-well microplates from laboratory video. Plate facts: 8 rows A-H (top to bottom); 12 columns 1-12 (left to right); well A1 is one corner, H12 diagonally opposite. To orient the plate, read any printed edge labels (A-H, 1-12) if legible; otherwise use the notched/beveled A1 corner. Pipette types: single $\rightarrow$ ONE well; 8-channel $\rightarrow$ a full COLUMN (8 wells, e.g. A5..H5); 12-channel $\rightarrow$ a full ROW (12 wells, e.g. C1..C12). A "dispensing"/commit moment is when the tip is lowered into a well and releasing liquid - not approaching, hovering above, or retracting.

Baseline A — single call (16 FPV frames).

You are shown 16 time-ordered frames of a single pipetting action, captured from an FPV camera, spanning approach through dispensing to retraction. Determine which well(s) the pipette dispenses into. Reason step by step:

1. **ORIENTATION** - state which on-screen direction is increasing column (1 $\rightarrow$ 12) and which is increasing row (A $\rightarrow$ H), and how you determined it (printed labels or beveled A1 corner).
2. **COMMIT MOMENT** - identify which frame shows the tip lowered into a well and dispensing.
3. **TIP LOCATION** - count columns from the left reference edge and rows from the top reference edge to the tip.
4. **PIPETTE TYPE** - single / 8-channel (full column) / 12-channel (full row).
5. **ANSWER** - report the pipette_type and the (well_row, well_column) of the well the tip is centered over (any one well in the dispensed row/column if multi-channel).

Baseline B — Call 1: select dispensing frame (16 FPV frames).

You are shown 16 time-ordered frames (indices 0-15) of a single pipetting action from an FPV camera. Identify the index of the frame that best shows the pipette **ACTIVELY DISPENSING** - tip lowered into a well, releasing liquid - as opposed to approaching, hovering, or retracting. Reason briefly, then give the index.

Baseline B — Call 2: read well (one full-resolution FPV frame).

This is the single dispensing moment, shown at full resolution from an FPV camera. Determine which well(s) the pipette is dispensing into. Reason step by step:

1. **ORIENTATION** - state the +column (1 $\rightarrow$ 12) and +row (A $\rightarrow$ H) directions and how you determined them.
 2. **TIP LOCATION** - count columns from the left edge and rows from the top edge to the tip.
 3. **PIPETTE TYPE** - single / 8-channel / 12-channel.
- ANSWER** - report the pipette_type and the (well_row, well_column) the tip is over (any one well in the dispensed row/column if multi-channel).